

Day 10 - Fibonacci Series ☐

Question

Fibonacci Series print karo.

Fibonacci Series Kya Hai?

Har number:

Previous 2 numbers ka sum

hota hai.

Example

0 1 1 2 3 5 8 13 21 34 ...

Explanation:

$$0 + 1 = 1$$

$$1 + 1 = 2$$

$$1 + 2 = 3$$

$$2 + 3 = 5$$

$$3 + 5 = 8$$

Approach 1 - Iterative (Best for Interviews)



Logic

2 variables rakho:

```
a = 0  
b = 1
```

Next term:

```
c = a + b
```

Fir values update karo.

Dry Run

```
a = 0  
b = 1
```

Print: 0 1

```
c = 0 + 1 = 1
```

Print: 1

```
a = 1  
b = 1
```

```
c = 1 + 1 = 2
```

Print: 2

Java Code

```
public class Day10 {  
    public static void main(String[] args) {  
        int n = 10;  
  
        int a = 0;  
        int b = 1;  
  
        System.out.print(a + " " + b + " ");  
  
        for(int i = 3; i <= n; i++) {  
            int c = a + b;  
  
            System.out.print(c + " ");  
  
            a = b;  
            b = c;  
        }  
    }  
}
```

Output

0 1 1 2 3 5 8 13 21 34

Time Complexity

$O(N)$

Space Complexity

$O(1)$

Approach 2 - Recursion

Logic

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

Example

$\text{fib}(5)$
 $\text{fib}(4) + \text{fib}(3)$
 $=$
 5

Java Code

```
public class Day10 {  
    static int fib(int n) {  
        if(n <= 1)  
            return n;  
        return fib(n - 1) + fib(n - 2);  
    }  
    public static void main(String[] args) {  
        int n = 10;  
        for(int i = 0; i < n; i++) {  
            System.out.print(fib(i) + " ");  
        }  
    }  
}
```

Time Complexity

$O(2^N)$

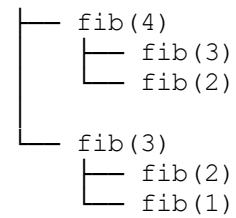
Space Complexity

$O(N)$

Why Recursive Approach is Slow?

Example

`fib(5)`



Observe:

`fib(3)`
`fib(2)`

baar-baar calculate ho rahe hain.

Ye unnecessary work hai.

Approach 3 - Dynamic Programming (Memoization)

Logic

Ek baar calculate karo.

Store kar lo.

Dobara calculate mat karo.

Java Code

```
import java.util.Arrays;

public class Day10 {

    static int[] dp = new int[100];

    static int fib(int n) {

        if(n <= 1)
            return n;

        if(dp[n] != -1)
            return dp[n];

        return dp[n] =
            fib(n - 1) +
            fib(n - 2);
    }

    public static void main(String[] args) {

        Arrays.fill(dp, -1);

        System.out.println(fib(10));
    }
}
```

Time Complexity

$O(N)$

Space Complexity

$O(N)$

Interview Follow-Up Questions

Q1. First Fibonacci Number?

Different books me alag conventions hain.

Most interviews:

0 1 1 2 3 5 ...

follow karte hain.

Q2. Best Solution?

Iterative

Time : $O(N)$
Space : $O(1)$

Sabse efficient.

Q3. Why Recursion Not Preferred?

Repeated Calculations
High Memory Usage
Slow Execution

Interview Concept of the Day

Dynamic Programming (DP)

DP ka golden rule:

Overlapping Subproblems
+
Store Previous Results

Fibonacci DP ka sabse famous example hai.

Future me:

- Climbing Stairs
- Knapsack
- Longest Common Subsequence
- DP on Trees

sab isi concept par based hain.

Summary

Approach	Time	Space
Iterative ☐	$O(N)$	$O(1)$
Recursion	$O(2^N)$	$O(N)$
DP Memoization	$O(N)$	$O(N)$

Practice Questions

Find:

Fib(5)
Fib(6)
Fib(7)
Fib(8)
Fib(10)

Answers khud calculate karo.

Interview Trick

Agar interviewer bole:

"100th Fibonacci Number nikalo"

To simple recursion mat use karna.

Bolna:

Recursion = $O(2^N)$

Iterative / DP = $O(N)$

Ye answer interviewer ko impress karta hai.